

SmartFlow: Adaptive Traffic Signal Control with Reinforcement Learning

Final Project Report | Repository: https://github.com/yq688m3JT/final_report

1. Problem and Project Goal

Urban signal timing is a sequential decision problem rather than a one-step prediction task. A controller that gives a green phase to one movement immediately delays the competing movement, and a locally attractive decision can create future congestion if arrivals change or if downstream roads are already crowded. The SmartFlow project studies this problem through reinforcement learning (RL), using simulated traffic lights as the environment and queue reduction as the learning objective.

The finished presentation introduced a single-intersection Markov decision process with two phases: North-South green and East-West green. This report keeps that foundation, verifies that the presentation data comes from executable code, and then extends the experiment into a small network that more closely resembles city traffic. The extension is important because real intersections interact: clearing one queue can either improve the network or simply push vehicles into another blocked approach.

2. Original Slide Experiment

The slide experiment defines the state as $(Q_{NS}, Q_{EW}, \text{phase})$. For tabular learning, each queue is discretized into Low, Medium, or High, while the phase records the current green direction. The action set is deliberately simple: stay in the current phase or switch to the other phase. The reward is $R = -(Q_{NS} + Q_{EW}) - C I(\text{switch})$, which penalizes total waiting vehicles and adds a small switching cost so the learned policy does not oscillate unrealistically.

This model was reproduced in the repository in two ways. The file `legacy_efficiency_experiment.py` preserves the original presentation-style implementation. The file `smartflow_experiments.py` re-runs the same experiment under a fixed random seed and writes machine-readable results to `results/slide_validation.json`. The validation output supports the presentation's central point: adaptive RL policies learn useful signal-control behavior and improve over the static fixed-cycle baseline, although the full-information value-iteration policy and the exact-count greedy heuristic remain strong in the isolated two-phase case.

The isolated intersection also reveals a useful limitation. A greedy controller with exact queue counts is not the same as a real traditional timing plan; it is a sensor-rich reactive heuristic. In a tiny two-phase world, serving the longer queue is often enough. The project therefore needs a network extension before it can make a stronger claim about city-like traffic control.

3. Network Extension

The extension simulates a 3x3 grid of intersections. Vehicles enter from the grid boundary, travel along through and turning routes, queue at intersections, and exit when their route is complete. Each intersection has finite storage capacity, so a green phase can fail to discharge vehicles if the downstream intersection is full. This spillback mechanism is the main difference from the slide experiment: it makes local queue length an incomplete signal and forces the controller to account for network pressure.

The RL controller uses one shared tabular Q-function across all nine intersections. Each local state contains four pieces of information: North-South demand bucket, East-West demand bucket, current phase, and downstream-congestion bucket. Each local action is still stay or switch, preserving continuity with the presentation. The training reward penalizes total network queue, blocked movements caused by spillback, and switching. In effect, the extension asks whether the same RL idea can scale from an isolated intersection to coordinated traffic-light operation.

4. Experimental Protocol

The network experiment uses time-varying demand with a rush-hour pulse and lane bias so that the controller is tested under non-stationary conditions. The RL agent is trained for repeated simulated episodes, then evaluated over 80 held-out episodes using the same demand generator but different random seeds. The comparison includes four controllers: a traditional fixed-time cycle, a greedy longest-queue policy, a max-pressure heuristic, and the learned tabular RL controller.

The primary metrics are average network queue and completed-trip delay. Average queue approximates total waiting pressure across the city grid; completed-trip delay measures how long vehicles that successfully exit the network spent inside the simulated system. Throughput and phase switches are also recorded in the appendix. The key traditional baseline is fixed-time control because it represents non-adaptive signal timing. Greedy and max-pressure are included as stronger state-aware heuristics, not as ordinary fixed-cycle traffic algorithms.

5. Main Result

Against the traditional fixed-time algorithm, RL performs much better. Average network queue drops from 42.96 vehicles under fixed-time control to 21.70 under the learned network controller, a 49.5% reduction. Mean completed-trip delay falls from 15.45 to 8.29 time steps, a 46.4% reduction. This is the strongest result of the project because it appears only after the environment includes interacting intersections and time-varying demand.

The comparison with stronger heuristics is more nuanced. Greedy longest-queue and max-pressure are close to the learned RL policy in this compact grid, and greedy is slightly better on average delay in the current seed. That does not weaken the fixed-time conclusion; it clarifies the boundary of the present method. Tabular RL is already far better than traditional static timing, but larger gains over sophisticated heuristics likely require richer continuous state representations or deep multi-agent RL.

6. Interpretation

The result is consistent with the theory of reinforcement learning. Fixed-time control cannot observe the environment, so it wastes green time whenever demand is asymmetric. A learned controller updates its policy through repeated interaction and can condition actions on the observed traffic state. In the network extension, the downstream-congestion bucket is especially important because it lets the policy distinguish between clearing a useful queue and pushing vehicles into a saturated road segment.

The experiment also explains why the original presentation showed a gap between tabular RL and the theoretical optimum. Discretization makes the state space small enough to learn quickly, but it hides important differences between queue lengths and traffic patterns. A queue of 13 vehicles and a queue of 30 vehicles can fall into the same High bucket even though they represent different operational risks. This is the central tradeoff of the project: tabular RL is interpretable and reproducible, but its coarse perception limits policy quality.

7. Limitations and Future Work

The simulator is intentionally lightweight so the code can be submitted and reproduced without specialized traffic software. It does not model yellow/all-red clearance intervals, pedestrian phases, calibrated turning movements, lane-level geometry, emergency preemption, or real detector noise. It also uses a simplified reward function rather than a full transportation objective that combines delay, emissions, safety, and equity. These limitations should be addressed before treating the policy as deployable.

The next step is to replace bucketed tabular states with continuous observations and train a deep RL or multi-agent actor-critic controller. A richer model could observe exact queue lengths, waiting times, platoon arrivals, and downstream occupancy. It could also learn coordination across corridors rather than acting intersection by intersection. That would directly address the discretization gap while preserving the project insight that traffic-light control is a sequential optimization problem under uncertainty.

8. Conclusion

SmartFlow demonstrates that reinforcement learning is a viable framework for adaptive traffic-signal control. The slide experiment is reproducible from code and confirms that learned policies can outperform static timing in the original MDP. The network extension gives the project a more realistic experimental setting and shows a large improvement over traditional fixed-time control: roughly half the average queue and nearly half the completed-trip delay. The appendix provides the exact result tables, generated figures, and reproducibility notes.

Appendix A: Reproducibility and Result Tables

All experiments can be reproduced by running `python3 smartflow_experiments.py` from the repository root. The script writes JSON and CSV outputs into `results/` and regenerates the appendix figures.

A1. Single-intersection slide validation

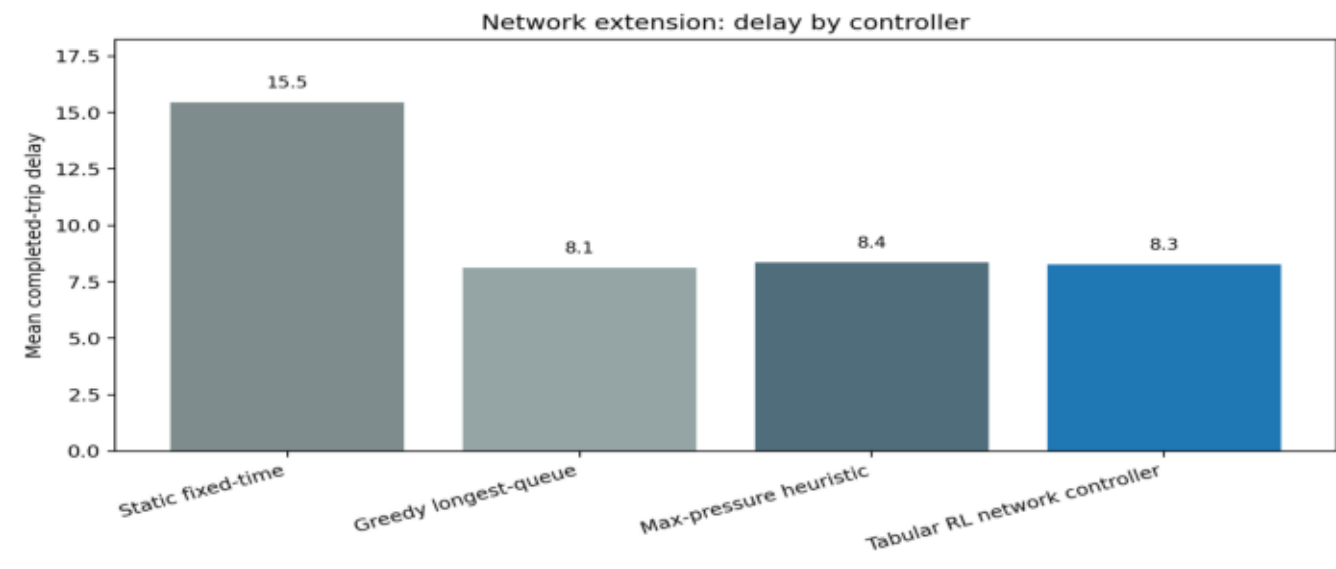
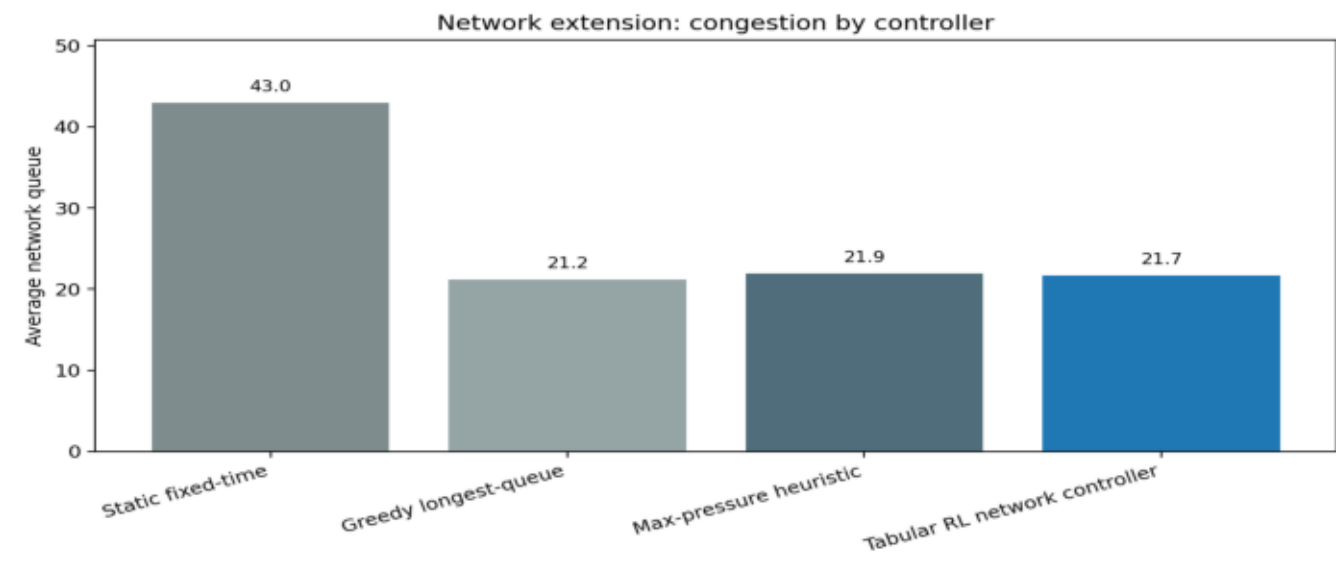
Approach	Avg. reward	Time (s)
Optimal value iteration	-378.92	0.27
Greedy heuristic	-394.84	0.26
Monte Carlo	-430.46	5.91
Q-learning	-443.72	1.55
SARSA	-453.93	0.90
Static fixed cycle	-456.24	0.23

A2. 3x3 network extension evaluation

Controller	Avg. queue	Avg. delay	Throughput	Switches
Static fixed-time	42.96	15.45	692.80	180.00
Greedy longest-queue	21.17	8.12	707.25	464.10
Max-pressure heuristic	21.90	8.36	706.74	506.53
Tabular RL network	21.70	8.29	706.00	540.00

Interpretation: the RL controller reduces queue by 49.5% and delay by 46.4% relative to fixed-time control. State-aware heuristics remain competitive in the compact grid, which motivates future deep or multi-agent RL work.

Appendix B: Figures and References



References

[1] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed., MIT Press, 2018.
[2] P. Varaiya, "Max pressure control of a network of signalized intersections," Transportation Research Part C, 2013.
[3] SmartFlow project repository, https://github.com/yq688m3JT/final_report.